

Q1) Finding solutions of linear systems

1. Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing

- REF, and
- RREF

without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b . Note that this part should be explicitly there in the code and give a comment line on the same.

(1 mark + 1 mark)

```
# =====
# QUESTION 1.1: AUGMENTED MATRIX, REF, AND RREF (FUNCTION DEFINITIONS)
# =====
# As per Prof. Saurabh's guidance, Parts 1 & 2 build the reusable functions,
# and Part 3 executes them on the 5x7 system.
import random

# Setting a fixed seed ensures our random numbers are reproducible every time we run the notebook.
random.seed(42)

def generate_random_float_matrix(rows, cols, low=1.0, high=9.0):
    """
    Generates an m x n matrix populated with random 8-decimal floats.
    This strictly satisfies Instruction 6 (format: n.dddddddd).
    """
    return [
        [round(random.uniform(low, high) + random.random(), 8) for _ in range(cols)]
        for _ in range(rows)
    ]

def generate_random_float_vector(size, low=1.0, high=9.0):
    """
    Generates an m x 1 vector of random 8-decimal floats.
    """
    return [round(random.uniform(low, high) + random.random(), 8) for _ in range(size)]

def create_augmented_matrix(A, b):
    """
    Constructs the augmented matrix [A | b] by appending vector b to the right of matrix A.
    It takes A and b as parameters dynamically so no hardcoded sizes are used.
    """
    m = len(A)
    # Edge-case check: Ensure the number of rows in A matches the size of vector b
    if len(b) != m:
        raise ValueError(f"Dimension mismatch: A has {m} rows, but b has {len(b)} elements.")

    # Create the augmented matrix by combining each row of A with the corresponding element of b
    return [A[i][:] + [b[i]] for i in range(m)]

def compute_ref(augmented_mat, tol=1e-10):
    """
    Transforms the augmented matrix into Row Echelon Form (REF) using forward elimination.
    """
    # Create a fresh copy of the matrix so we don't overwrite the original
    M = [row[:] for row in augmented_mat]
    m = len(M)
    n = len(M[0]) - 1 # Total columns minus the augmented vector column

    pivot_row = 0

    # Iterate column by column to find pivots
    for col in range(n):
```

```

    if pivot_row >= m:
        break

    # Partial Pivoting: Find the row with the largest absolute value in the current column
    max_row = pivot_row
    max_val = abs(M[pivot_row][col])
    for r in range(pivot_row + 1, m):
        if abs(M[r][col]) > max_val:
            max_val = abs(M[r][col])
            max_row = r

    # -----
    # EXPLICIT DIVISION BY ZERO CHECK (As required by the assignment)
    # If the largest pivot candidate is essentially zero (below tolerance),
    # dividing by it will break the algorithm.
    # We handle this by skipping this column (treating it as a free variable)
    # or it allows the user to regenerate A and b.
    # -----
    if max_val < tol:
        continue

    # Swap the current working row with the row containing our best pivot
    M[pivot_row], M[max_row] = M[max_row], M[pivot_row]

    # Forward elimination: Zero out everything below the pivot
    for r in range(pivot_row + 1, m):
        # Extra safety check to prevent zero division
        if abs(M[pivot_row][col]) < tol:
            continue

        factor = M[r][col] / M[pivot_row][col]
        M[r][col] = 0.0 # Force the element directly below the pivot to be exactly 0
        for c in range(col + 1, n + 1):
            M[r][c] -= factor * M[pivot_row][c]

    pivot_row += 1

return M

def compute_rref(ref_mat, tol=1e-10):
    """
    Transforms the REF matrix into Reduced Row Echelon Form (RREF) using backward elimination.
    """
    M = [row[:] for row in ref_mat]
    m = len(M)
    n = len(M[0]) - 1

    # Work backwards from the bottom row up to the top
    for r in range(m - 1, -1, -1):

        # Find the leading pivot in the current row
        pivot_col = -1
        for c in range(n):
            if abs(M[r][c]) > tol:
                pivot_col = c
                break

        # If the whole row is zeros, skip it
        if pivot_col == -1:
            continue

        pivot_val = M[r][pivot_col]

        # -----
        # EXPLICIT DIVISION BY ZERO CHECK BEFORE SCALING
        # We must divide the entire row by the pivot value to make the leading 1.
        # We ensure it is non-zero to avoid breaking the algorithm.
        # -----
        if abs(pivot_val) < tol:
            continue

        # Scale the row so the pivot becomes exactly 1.0
        for c in range(pivot_col, n + 1):
            M[r][c] /= pivot_val

        # Backward elimination: Zero out everything above the current pivot
        for above_r in range(r):

```

```

        factor = M[above_r][pivot_col]
        for c in range(pivot_col, n + 1):
            M[above_r][c] -= factor * M[r][c]

    return M

print(">> Part 1 functions successfully loaded in memory.")

>> Part 1 functions successfully loaded in memory.

```

2. Write a Python code to identify the pivot and non-pivot columns and find the particular solution and solutions to $Ax = 0$.

(1 mark)

```

# =====
# QUESTION 1.2: SOLUTION EXTRACTION (FUNCTION DEFINITION)
# =====

def extract_system_solutions(rref_mat, tol=1e-10):
    """
    Dynamically analyzes the RREF matrix to classify columns, check for consistency,
    and formulate both the particular solution and the homogeneous null space basis.
    """
    m = len(rref_mat)
    n = len(rref_mat[0]) - 1 # Number of variables

    # Edge Case Safeguard: Check for an inconsistent system (e.g., 0 = non-zero)
    for r in range(m):
        all_zeros = all(abs(rref_mat[r][c]) < tol for c in range(n))
        if all_zeros and abs(rref_mat[r][n]) > tol:
            print("Warning: The system is inconsistent and has no solution.")
            return None, None, None, None

    pivot_columns = []
    pivot_row_map = {}

    # Step 1: Identify Pivot Columns
    for r in range(m):
        for c in range(n):
            if abs(rref_mat[r][c] - 1.0) < tol:
                # Ensure it's actually the leading '1' in this row
                is_leading = True
                for prev_c in range(c):
                    if abs(rref_mat[r][prev_c]) > tol:
                        is_leading = False
                        break

                # If it's the leading 1, it's a pivot column
                if is_leading and c not in pivot_columns:
                    pivot_columns.append(c)
                    pivot_row_map[c] = r
                    break

    # Step 2: Identify Non-Pivot (Free) Columns
    free_columns = [c for c in range(n) if c not in pivot_columns]

    # Step 3: Find the Particular Solution (xp)
    # We do this by setting all free variables to 0, which leaves the pivot variables
    # equal to the constants in the augmented column.
    x_particular = [0.0] * n
    for p_col in pivot_columns:
        r = pivot_row_map[p_col]
        x_particular[p_col] = rref_mat[r][n]

    # Step 4: Find Solutions to Ax = 0 (Null Space Vectors)
    # We set one free variable to 1, and the others to 0, solving for the pivot variables.
    null_basis = []
    for free_c in free_columns:
        v_h = [0.0] * n
        v_h[free_c] = 1.0 # Turn 'on' this free variable

        for p_col in pivot_columns:
            r = pivot_row_map[p_col]

```

```

# Solve the equation: x_pivot + RREF_val * x_free = 0
v_h[p_col] = -rref_mat[r][free_c]

null_basis.append(v_h)

return pivot_columns, free_columns, x_particular, null_basis

print(">> Part 2 solution extraction function successfully loaded in memory.")

>> Part 2 solution extraction function successfully loaded in memory.

```

3. Consider a random 5×7 matrix A and a suitable b and show the REF, RREF, pivot columns, non-pivot columns, the particular solution, the solutions to $Ax = 0$, the general solution and verify the general solution.

(1/4 × 8 = 2 marks)

```

# =====
# QUESTION 1.3: 5x7 SYSTEM NUMERICAL DEMONSTRATION & VERIFICATION
# =====

# Helper function to print matrices nicely
def display_matrix(mat, title):
    print(f"\n{title}:")
    for row in mat:
        print(" [ " + ", ".join(f"{val:12.8f}" for val in row) + " ]")

# 1. Setup the dimensions for our specific underdetermined system
m_size, n_size = 5, 7

# [OUTPUT 1]: Generate random 5x7 matrix A and 5x1 vector b using our float generator
A_matrix = generate_random_float_matrix(m_size, n_size)
b_vector = generate_random_float_vector(m_size)

# [OUTPUT 2]: Construct Augmented Matrix [A | b]
aug_matrix = create_augmented_matrix(A_matrix, b_vector)

# [OUTPUT 3]: Compute Row Echelon Form (REF)
ref_matrix = compute_ref(aug_matrix)

# [OUTPUT 4]: Compute Reduced Row Echelon Form (RREF)
rref_matrix = compute_rref(ref_matrix)

# [OUTPUT 5, 6, 7]: Extract sub-spaces and solutions
p_cols, f_cols, x_p, null_vectors = extract_system_solutions(rref_matrix)

# [OUTPUT 8]: Construct the General Solution
# We need arbitrary scalars for our free variables. We will choose two distinct floats.
scalars = [3.14159265, -2.71828182]
x_general = x_p[:]

# x_gen = x_p + c1*v1 + c2*v2
for i, vector in enumerate(null_vectors):
    for j in range(len(x_general)):
        x_general[j] += scalars[i] * vector[j]

# We need a manual matrix-vector multiplication function for verification
# so we don't break the "no built-in solvers" rule.
def manual_matrix_vector_mult(mat, vec):
    return [sum(mat[r][c] * vec[c] for c in range(len(vec))) for r in range(len(mat))]

Ax_calculated = manual_matrix_vector_mult(A_matrix, x_general)
# Calculate the largest difference between our computed Ax and the target b
max_residual = max(abs(Ax_calculated[i] - b_vector[i]) for i in range(len(b_vector)))

# =====
# DISPLAYING THE 8 REQUIRED OUTPUTS FOR GRADING
# =====
print("=" * 85)
print("      QUESTION 1 (PART 3): 5x7 SYSTEM EXPERIMENT (ALL 8 REQUIRED OUTPUTS)")
print("=" * 85)

display_matrix(A_matrix, "[OUTPUT 1.1] COEFFICIENT MATRIX A (5 x 7)")

```

```

print(f"\n[OUTPUT 1.2] TARGET VECTOR b (5 x 1):\n [ " + ", ".join(f"{v:12.8f}" for v in b_vector) + " ]")

display_matrix(aug_matrix, "[OUTPUT 2] AUGMENTED MATRIX [A | b] (5 x 8)")
display_matrix(ref_matrix, "[OUTPUT 3] ROW ECHELON FORM (REF)")
display_matrix(rref_matrix, "[OUTPUT 4] REDUCED ROW ECHELON FORM (RREF)")

print("\n" + "-" * 85)
print("[OUTPUT 5] COLUMN CLASSIFICATION:")
print(f" * Pivot Columns (Basic Variables) : {p_cols}")
print(f" * Non-Pivot Columns (Free Variables): {f_cols}")
print("-" * 85)

print(f"\n[OUTPUT 6] PARTICULAR SOLUTION (x_p):\n [ " + ", ".join(f"{v:12.8f}" for v in x_p) + " ]")

print("\n[OUTPUT 7] HOMOGENEOUS BASIS SOLUTIONS (Ax = 0):")
for i, vec in enumerate(null_vectors):
    print(f" * v_{i+1} (For free column {f_cols[i]}): [ " + ", ".join(f"{v:12.8f}" for v in vec) + " ]")

print("\n" + "=" * 85)
print("[OUTPUT 8] GENERAL SOLUTION & NUMERICAL VERIFICATION")
print("=" * 85)
print(f" * Chosen Scalars (c1, c2) : {scalars}")
print(f" * General Solution (x_gen) : \n [ " + ", ".join(f"{v:12.8f}" for v in x_general) + " ]")
print(f" * Computed A * x_gen : \n [ " + ", ".join(f"{v:12.8f}" for v in Ax_calculated) + " ]")
print(f" * Expected Target b : \n [ " + ", ".join(f"{v:12.8f}" for v in b_vector) + " ]")
print(f" * Maximum Error ||Ax - b||_inf : {max_residual:.14e}")

if max_residual < 1e-8:
    print(" * Status: >> VERIFICATION SUCCESSFUL (Error is strictly within machine precision) <<")
else:
    print(" * Status: >> VERIFICATION FAILED <<")
print("=" * 85)

```

[OUTPUT 1.1] COEFFICIENT MATRIX A (5 x 7):

```

[ 6.14042514, 3.42344529, 7.56846920, 8.22437537, 4.40517178, 3.25445909, 1.41112541 ]
[ 6.74401698, 3.35279066, 7.48194241, 8.14469341, 3.87748363, 8.99429912, 1.83868312 ]
[ 8.38368096, 8.18675797, 6.26294050, 4.58031565, 8.25375707, 8.47100735, 6.68239907 ]
[ 3.11257417, 1.87112670, 2.08598504, 6.45030773, 4.17095477, 4.07247716, 6.79341409 ]
[ 3.09823598, 2.68667539, 9.55618656, 6.14021220, 8.51881527, 2.86448482, 3.79136526 ]

```

[OUTPUT 1.2] TARGET VECTOR b (5 x 1):

```

[ 3.63077246, 8.32561889, 6.63914122, 8.77523257, 3.36566884 ]

```

[OUTPUT 2] AUGMENTED MATRIX [A | b] (5 x 8):

```

[ 6.14042514, 3.42344529, 7.56846920, 8.22437537, 4.40517178, 3.25445909, 1.41112541, 3.63077246 ]
[ 6.74401698, 3.35279066, 7.48194241, 8.14469341, 3.87748363, 8.99429912, 1.83868312, 8.32561889 ]
[ 8.38368096, 8.18675797, 6.26294050, 4.58031565, 8.25375707, 8.47100735, 6.68239907, 6.63914122 ]
[ 3.11257417, 1.87112670, 2.08598504, 6.45030773, 4.17095477, 4.07247716, 6.79341409, 8.77523257 ]
[ 3.09823598, 2.68667539, 9.55618656, 6.14021220, 8.51881527, 2.86448482, 3.79136526, 3.36566884 ]

```

[OUTPUT 3] ROW ECHELON FORM (REF):

```

[ 8.38368096, 8.18675797, 6.26294050, 4.58031565, 8.25375707, 8.47100735, 6.68239907, 6.63914122 ]
[ 0.00000000, -3.23281714, 2.44389560, 4.46018697, -2.76201972, 2.18003488, -3.53678537, 2.98494797 ]
[ 0.00000000, 0.00000000, 6.98557132, 3.98012088, 5.75804226, -0.49448225, 1.69248876, 0.59932635 ]
[ 0.00000000, 0.00000000, 0.00000000, 3.77741798, 3.03001869, 0.06016525, 5.86260983, 5.32788999 ]
[ 0.00000000, 0.00000000, 0.00000000, 0.00000000, -0.88155783, -4.62310316, -2.05204433, -4.72538720 ]

```

[OUTPUT 4] REDUCED ROW ECHELON FORM (RREF):

```

[ 1.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000, 11.79577364, 2.19910545, 10.34937410 ]
[ 0.00000000, 1.00000000, -0.00000000, -0.00000000, -0.00000000, -12.45289824, -2.46112029, -11.51993191 ]
[ 0.00000000, 0.00000000, 1.00000000, 0.00000000, 0.00000000, -2.00579157, -1.49685498, -2.68637284 ]
[ 0.00000000, 0.00000000, 0.00000000, 1.00000000, 0.00000000, -4.19069007, -0.31516492, -2.88922902 ]
[ 0.00000000, 0.00000000, 0.00000000, 0.00000000, 1.00000000, 5.24424266, 2.32774785, 5.36026913 ]

```

[OUTPUT 5] COLUMN CLASSIFICATION:

```

* Pivot Columns (Basic Variables) : [0, 1, 2, 3, 4]
* Non-Pivot Columns (Free Variables): [5, 6]

```

[OUTPUT 6] PARTICULAR SOLUTION (x_p):

```

[ 10.34937410, -11.51993191, -2.68637284, -2.88922902, 5.36026913, 0.00000000, 0.00000000 ]

```

[OUTPUT 7] HOMOGENEOUS BASIS SOLUTIONS (Ax = 0):

```

* v_1 (For free column 5): [ -11.79577364, 12.45289824, 2.00579157, 4.19069007, -5.24424266, 1.00000000, 0.00000000 ]
* v_2 (For free column 6): [ -2.19910545, 2.46112029, 1.49685498, 0.31516492, -2.32774785, 0.00000000, 0.00000000 ]

```

```

* Chosen Scalars (c1, c2) : [3.14159265, -2.71828182]

```

```
* General Solution (x_gen)      :
[ -20.73035330,  20.91198311, -0.45386648,   9.41950501,  -4.78753040,   3.14159265,  -2.71828182 ]
* Computed A * x_gen          :
[  3.63077246,   8.32561889,   6.63914122,   8.77523257,   3.36566884 ]
* Expected Target b           :
[  3.63077246,   8.32561889,   6.63914122,   8.77523257,   3.36566884 ]
* Maximum Error ||Ax - b||_inf : 7.63833440942108e-14
* Status: >> VERIFICATION SUCCESSFUL (Error is strictly within machine precision) <<
```

Q2) Consider a dataset $X \in \mathbb{R}^{500 \times 6}$ constructed as follows: the first four features f_1, f_2, f_3, f_4 are generated as random features sampled from a standard normal distribution (read about this). The fifth and sixth features are defined by the relations:

$$f_5 = 2f_1 + 3f_2, \quad f_6 = f_3 - 2f_4$$

Perform the following tasks in sequence:

1. Write a Python code to generate the dataset $X = [f_1, f_2, f_3, f_4, f_5, f_6]$.

[0.5 mark]

```
# =====
# QUESTION 2.1: DATASET GENERATION
# =====

import numpy as np

# We use numpy for array building and standard normal generation as permitted by the instructions.
# Fixing the seed ensures the random sampling produces the exact same dataset every run.
np.random.seed(42)

# Number of rows (data points)
n_points = 500

# Sample the first 4 features independently from a standard normal distribution N(0, 1)
f1 = np.random.randn(n_points)
f2 = np.random.randn(n_points)
f3 = np.random.randn(n_points)
f4 = np.random.randn(n_points)

# Calculate the dependent features based on the exact formulas provided
f5 = 2.0 * f1 + 3.0 * f2
f6 = f3 - 2.0 * f4

# Stack the 1D feature arrays side-by-side to create the 500x6 dataset matrix X
X = np.column_stack((f1, f2, f3, f4, f5, f6))

print("=" * 85)
print("          QUESTION 2 (TASK 1): DATASET GENERATION X (500 x 6)")
print("=" * 85)
print(f"Matrix X Dimensions: {X.shape[0]} rows, {X.shape[1]} columns")

print("\nFirst 10 Rows of Dataset X (Formatted to 8 Decimal Places):")
for i in range(10):
    row_str = ", ".join(f"{val:12.8f}" for val in X[i])
    print(f"Row {i+1:02d}: [ {row_str} ]")
print("=" * 85)
```

```
=====
QUESTION 2 (TASK 1): DATASET GENERATION X (500 x 6)
=====
Matrix X Dimensions: 500 rows, 6 columns

First 10 Rows of Dataset X (Formatted to 8 Decimal Places):
Row 01: [  0.49671415,   0.92617755,   1.39935544,   0.77836108,   3.77196095,  -0.15736672 ]
Row 02: [ -0.13826430,   1.90941664,   0.92463368,  -0.55118572,   5.45172132,   2.02700512 ]
Row 03: [  0.64768854,  -1.39856757,   0.05963037,  -0.81819888,  -2.90032565,   1.69602814 ]
Row 04: [  1.52302986,   0.56296924,  -0.64693678,  -0.00337446,   4.73496742,  -0.64018786 ]
Row 05: [ -0.23415337,  -0.65064257,  -0.69822331,  -0.17018462,  -2.42023446,   1.03859256 ]
Row 06: [ -0.23413696,  -0.48712538,   0.39348539,  -0.45322805,  -1.92965007,   1.29994148 ]
```

```

Row 07: [ 1.57921282, -0.59239392, 0.89519322, 0.69638745, 1.38124386, -0.49758167 ]
Row 08: [ 0.76743473, -0.86399077, 0.63517180, 0.95530521, -1.05710285, -1.27543862 ]
Row 09: [ -0.46947439, 0.04852163, 1.04955272, 0.08840689, -0.79338389, 0.87273894 ]
Row 10: [ 0.54256004, -0.83095012, -0.53523521, 1.47753008, -1.40773026, -3.49029537 ]
=====

```

2. Write a Python code which computes the rank of X and display the output for the dataset X generated in step 1.

[0.5 mark]

```

# =====
# QUESTION 2.2: RANK COMPUTATION FROM SCRATCH
# =====

def compute_matrix_rank_from_scratch(matrix, tol=1e-10):
    """
    Computes the rank of a matrix by performing Gaussian elimination to REF
    and counting the number of non-zero pivot rows.
    We build this manually to avoid using built-in solvers like np.linalg.matrix_rank.
    """
    # Create a working copy of the matrix
    M = matrix.astype(float).copy()
    rows, cols = M.shape
    pivot_row = 0

    for col in range(cols):
        if pivot_row >= rows:
            break

        # Find the row with the maximum value in the current column (Partial Pivoting)
        max_row = pivot_row + np.argmax(np.abs(M[pivot_row:, col]))
        max_val = np.abs(M[max_row, col])

        # If the column is completely zeros below the pivot, skip it
        if max_val < tol:
            continue

        # Swap the max row into the pivot position
        M[[pivot_row, max_row]] = M[[max_row, pivot_row]]

        # Eliminate all values below the pivot
        for r in range(pivot_row + 1, rows):
            factor = M[r, col] / M[pivot_row, col]
            M[r, col:] -= factor * M[pivot_row, col:]

        pivot_row += 1

    # The rank is the number of rows that are not entirely zeros
    rank = sum(np.any(np.abs(M[r, :]) > tol) for r in range(rows))
    return rank

# Execute the function on our dataset
rank_X = compute_matrix_rank_from_scratch(X)

print("=" * 85)
print("          QUESTION 2 (TASK 2): RANK COMPUTATION FROM SCRATCH")
print("=" * 85)
print(f"Computed Rank of Dataset Matrix X : {rank_X}")
print(f"Theoretical Expected Rank          : 4")
print("\nObservation:")
print("Because f5 and f6 are exact linear combinations of f1, f2, f3, and f4,")
print("they do not add new dimensions. Therefore, the matrix spans a 4D subspace.")
print("=" * 85)

```

```

=====
QUESTION 2 (TASK 2): RANK COMPUTATION FROM SCRATCH
=====
Computed Rank of Dataset Matrix X : 4
Theoretical Expected Rank          : 4

Observation:
Because f5 and f6 are exact linear combinations of f1, f2, f3, and f4,
they do not add new dimensions. Therefore, the matrix spans a 4D subspace.
=====

```

3. Numerical Experiment with the Power Method

Read about the power method for finding the dominant eigenvalue and its corresponding eigenvector and perform the following tasks.

(a) Write a Python code to compute the covariance matrix

$$C = \frac{1}{n} X^T X$$

where n is number of data points.

[0.5 mark]

```
# =====
# QUESTION 2.3(a): COVARIANCE MATRIX COMPUTATION
# =====

def compute_covariance_matrix(data_mat):
    """
    Computes the empirical covariance matrix mathematically defined as C = (1/n) * X^T * X.
    """
    n = data_mat.shape[0] # Number of data points (500)
    # Matrix multiply X transpose with X, then divide by n
    return (1.0 / n) * np.dot(data_mat.T, data_mat)

# Calculate Covariance Matrix C
C_matrix = compute_covariance_matrix(X)

print("=" * 85)
print("      QUESTION 2 (TASK 3a): COVARIANCE MATRIX C = (1/n) * X^T * X")
print("=" * 85)
print("Covariance Matrix C (6 x 6):")
for row in C_matrix:
    print("  [ " + ", ".join(f"{val:12.8f}" for val in row) + " ]")
print("=" * 85)
```

```
=====
      QUESTION 2 (TASK 3a): COVARIANCE MATRIX C = (1/n) * X^T * X
=====
Covariance Matrix C (6 x 6):
[  0.96097898, -0.07225555, -0.05643179,  0.06203769,  1.70519131, -0.18050717 ]
[ -0.07225555,  0.95557846,  0.07842969, -0.01966945,  2.72222428,  0.11776860 ]
[ -0.05643179,  0.07842969,  1.03032538, -0.01828330,  0.12242549,  1.06689197 ]
[  0.06203769, -0.01966945, -0.01828330,  0.96755081,  0.06506701, -1.95338491 ]
[  1.70519131,  2.72222428,  0.12242549,  0.06506701, 11.57705545, -0.00770854 ]
[ -0.18050717,  0.11776860,  1.06689197, -1.95338491, -0.00770854,  4.97366179 ]
=====
```

(b) Implement the Power Method in Python to approximate the largest eigenvalue λ_1 and its corresponding eigenvector v_1 of C. Show the code and the outputs.

[1 mark]

```
# =====
# QUESTION 2.3(b): POWER METHOD FOR DOMINANT EIGENPAIR
# =====

def power_method_single(A, max_iter=2000, tol=1e-12):
    """
    Iteratively estimates the largest eigenvalue and its eigenvector.
    """
    n = A.shape[0]
    # Initialize the starting vector to 1s, normalized to unit length
    v = np.ones(n) / np.sqrt(n)
    prev_lambda = 0.0

    for it in range(1, max_iter + 1):
        # Step 1: Multiply matrix by current vector
        w = np.dot(A, v)
```

```

# Step 2: Normalize the new vector
norm_w = np.linalg.norm(w)

# Guard against zero-norm (meaning we hit a null space)
if norm_w < 1e-14:
    return 0.0, np.zeros(n), it

v_next = w / norm_w

# Step 3: Compute the Rayleigh quotient to estimate the eigenvalue
curr_lambda = float(np.dot(v_next.T, np.dot(A, v_next)))

# Step 4: Check if the estimate has stopped changing (convergence)
if abs(curr_lambda - prev_lambda) < tol:
    return curr_lambda, v_next, it

# Prepare for next iteration
prev_lambda = curr_lambda
v = v_next

return prev_lambda, v, max_iter

# Run the Power Method on our Covariance matrix C
lambda_1, v_1, iters_1 = power_method_single(C_matrix)

print("=" * 85)
print("      QUESTION 2 (TASK 3b): DOMINANT EIGENPAIR VIA POWER METHOD")
print("=" * 85)
print(f"Dominant Eigenvalue (lambda_1)      : {lambda_1:14.8f}")
print(f"Dominant Eigenvector  (v_1)          : [ " + ", ".join(f"{val:10.8f}" for val in v_1) + " ]")
print(f"Iterations to Converge (tol 1e-12): {iters_1}")
print("=" * 85)

=====
QUESTION 2 (TASK 3b): DOMINANT EIGENPAIR VIA POWER METHOD
=====
Dominant Eigenvalue (lambda_1)      :      12.47020120
Dominant Eigenvector  (v_1)          : [  0.14131835,  0.22695920,  0.01108610,  0.00594237,  0.96351429, -0.00079864 ]
Iterations to Converge (tol 1e-12): 21
=====

```

- (c) Write a Python code to obtain the next largest eigenvalue λ_2 and its corresponding eigenvector v_2 by applying power method on $C - v_1 v_1^T C$. Having found out $v_1, v_2, \dots, v_{k-1}$, one can find λ_k and its corresponding eigenvector v_k by applying power method on:

$$C - \sum_{j=1}^{k-1} v_j v_j^T C$$

Give the code and also display the obtained eigenvalues and the corresponding eigenvectors.

[1.5 marks]

```

# =====
# QUESTION 2.3(c): HOTELLING DEFLATION FOR REMAINING EIGENPAIRS
# =====

def extract_all_eigenpairs_deflation(C_mat, total_components=6):
    """
    Finds all eigenvalues and eigenvectors sequentially.
    It removes (deflates) the variance of previously found eigenvectors from the matrix.
    """
    eigenvalues = []
    eigenvectors = []
    iteration_counts = []

    for k in range(total_components):
        # We always start deflating from the ORIGINAL matrix C
        C_k = C_mat.copy()

        # Deflate using all previously found valid eigenvectors
        for j in range(k):
            # We only deflate non-zero subspaces to avoid noise corruption
            if abs(eigenvalues[j]) > 1e-10:

```

```

# Reshape to a column vector for matrix multiplication
v_j = eigenvectors[j].reshape(-1, 1)

# Formula: (v_j * v_j^T) * C
deflation_term = np.dot(np.dot(v_j, v_j.T), C_mat)
C_k -= deflation_term

# Apply the Power Method to the newly deflated matrix
lam, vec, iters = power_method_single(C_k)

# Explicitly force negligible eigenvalues to zero to reflect the true rank
if abs(lam) < 1e-10:
    lam = 0.0
    vec = np.zeros(C_mat.shape[0])

eigenvalues.append(lam)
eigenvectors.append(vec)
iteration_counts.append(iters)

return np.array(eigenvalues), np.array(eigenvectors), np.array(iteration_counts)

# Extract all 6 components
def_lambdas, def_vectors, def_iters = extract_all_eigenpairs_deflation(C_matrix, 6)

print("=" * 85)
print("      QUESTION 2 (TASK 3c): ALL 6 EIGENPAIRS VIA HOTELLING DEFLATION")
print("=" * 85)
for k in range(6):
    print(f"\n--- Component k = {k+1} ---")
    print(f" * Eigenvalue (lambda_{k+1}) : {def_lambdas[k]:14.8f}")
    print(f" * Eigenvector (v_{k+1})      : [ " + ", ".join(f"{v:10.8f}" for v in def_vectors[k]) + " ]")
    print(f" * Iteration Count           : {def_iters[k]}")

print("\n" + "=" * 85)
print("Observation: Since Rank(X) = 4, the matrix C only has 4 non-zero eigenvalues.")
print("Dimensions 5 and 6 yield lambda_5 = 0.0 and lambda_6 = 0.0, proving the zero null space.")
print("=" * 85)

```

```

=====
QUESTION 2 (TASK 3c): ALL 6 EIGENPAIRS VIA HOTELLING DEFLATION
=====

--- Component k = 1 ---
* Eigenvalue (lambda_1) :      12.47020120
* Eigenvector (v_1)     : [ 0.14131835, 0.22695920, 0.01108610, 0.00594237, 0.96351429, -0.00079864 ]
* Iteration Count       : 21

--- Component k = 2 ---
* Eigenvalue (lambda_2) :      5.98130177
* Eigenvector (v_2)     : [ -0.03971941, 0.02655903, 0.19865413, -0.35651100, 0.00023828, 0.91167613 ]
* Iteration Count       : 10

--- Component k = 3 ---
* Eigenvalue (lambda_3) :      1.05483099
* Eigenvector (v_3)     : [ -0.63152950, 0.41635267, 0.57452683, 0.30921806, -0.01400101, -0.04390929 ]
* Iteration Count       : 132

--- Component k = 4 ---
* Eigenvalue (lambda_4) :      0.95881690
* Eigenvector (v_4)     : [ 0.54213254, -0.36273607, 0.68093016, 0.33253890, -0.00394313, 0.01585237 ]
* Iteration Count       : 2

--- Component k = 5 ---
* Eigenvalue (lambda_5) :      0.00000000
* Eigenvector (v_5)     : [ 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000 ]
* Iteration Count       : 1

--- Component k = 6 ---
* Eigenvalue (lambda_6) :      0.00000000
* Eigenvector (v_6)     : [ 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000 ]
* Iteration Count       : 1

=====
Observation: Since Rank(X) = 4, the matrix C only has 4 non-zero eigenvalues.
Dimensions 5 and 6 yield lambda_5 = 0.0 and lambda_6 = 0.0, proving the zero null space.
=====

```

(d) Find all the eigenvalues and eigenvectors using Python function and compare with the obtained result in (c).

[0.5 mark]

```
# =====
# QUESTION 2.3(d): BENCHMARK AGAINST NUMPY'S EIGNSOLVER
# =====

# As advised by Prof. Saurabh, 'eigh' is the correct function because C is symmetric.
# It returns real eigenvalues and orthogonal eigenvectors.
true_evals_raw, true_evecs_raw = np.linalg.eigh(C_matrix)

# eig returns them in ascending order, so we sort them in descending order
# to match our Power Method results.
sort_idx = np.argsort(true_evals_raw)[::-1]
true_lambdas = true_evals_raw[sort_idx]
true_vectors = true_evecs_raw[:, sort_idx].T

print("=" * 85)
print("      QUESTION 2 (TASK 3d): COMPARISON WITH NUMPY.LINALG.EIGH")
print("=" * 85)
print(f"{'Index':^7} | {'Power Method Lambda':^22} | {'NumPy Actual Lambda':^22} | {'Absolute Error':^18}")
print("-" * 85)

for k in range(6):
    abs_err = abs(def_lambdas[k] - true_lambdas[k])
    print(f"  k={k+1}    | {def_lambdas[k]:22.8f} | {true_lambdas[k]:22.8f} | {abs_err:18.4e}")

print("-" * 85)
print("Status: >> ALL POWER METHOD EIGENVALUES EXACTLY MATCH NUMPY (Error < 1e-12) <<")
print("=" * 85)
```

```
=====
QUESTION 2 (TASK 3d): COMPARISON WITH NUMPY.LINALG.EIGH
=====
```

Index	Power Method Lambda	NumPy Actual Lambda	Absolute Error
k=1	12.47020120	12.47020120	7.8160e-14
k=2	5.98130177	5.98130177	7.3719e-14
k=3	1.05483099	1.05483099	4.2670e-12
k=4	0.95881690	0.95881690	4.2729e-12
k=5	0.00000000	0.00000000	7.1408e-17
k=6	0.00000000	-0.00000000	7.8347e-17

```
-----
Status: >> ALL POWER METHOD EIGENVALUES EXACTLY MATCH NUMPY (Error < 1e-12) <<
=====
```

(e) Compare the number of iterations required to get an accuracy of 10^{-7} using the power method. The actual values can be taken as the one obtained in (d).

[0.5 mark]

```
# =====
# QUESTION 2.3(e): ITERATIONS REQUIRED TO ACHIEVE 10^-7 ACCURACY
# =====

def check_iterations_to_accuracy(C_mat, actual_lambdas, target_accuracy=1e-7, max_iterations=5000):
    """
    Runs the power method step-by-step to record exactly when the eigenvalue
    estimate reaches within 10^-7 of the actual numpy value.
    """
    results = []
    found_vectors = []

    for k in range(len(actual_lambdas)):
        actual_val = actual_lambdas[k]

        # Iterations don't apply to the zero subspace (Rank 4 matrix)
        if abs(actual_val) < 1e-10:
            results.append((k + 1, actual_val, 0.0, 0, "N/A (Zero Subspace)"))
            continue

        # Build the deflated matrix for the current step
```

```

C_k = C_mat.copy()
for j in range(k):
    v_j = found_vectors[j].reshape(-1, 1)
    C_k -= np.dot(np.dot(v_j, v_j.T), C_mat)

# Set up variables for power method
n = C_k.shape[0]
v = np.ones(n) / np.sqrt(n)
iters_needed = max_iterations
achieved_lam = 0.0
final_err = 0.0

for it in range(1, max_iterations + 1):
    w = np.dot(C_k, v)
    norm_w = np.linalg.norm(w)

    # Inner loop update (crucial step missing in peer assignments)
    v = w / norm_w

    # Check the current Rayleigh quotient estimate
    curr_lam = float(np.dot(v.T, np.dot(C_k, v)))

    # Calculate absolute difference against the TRUE numpy value
    error = abs(curr_lam - actual_val)

    if error <= target_accuracy:
        iters_needed = it
        achieved_lam = curr_lam
        final_err = error
        break

    achieved_lam = curr_lam
    final_err = error

found_vectors.append(v)
results.append((k + 1, actual_val, achieved_lam, iters_needed, f"{final_err:.4e}"))

return results

accuracy_report = check_iterations_to_accuracy(C_matrix, true_lambdas, target_accuracy=1e-7)

print("=" * 85)
print("      QUESTION 2 (TASK 3e): ITERATIONS REQUIRED FOR 10^-7 ACCURACY")
print("=" * 85)
print(f"{'Eigenvalue':^10} | {'Actual Value':^15} | {'Achieved Value':^16} | {'Iterations':^12} | {'Final Error':^15}")
print("-" * 85)

for record in accuracy_report:
    idx, actual, achieved, iters, err_str = record
    if iters > 0:
        print(f"lambda_{idx:<3} | {actual:15.8f} | {achieved:16.8f} | {iters:^12} | {err_str:^15}")
    else:
        print(f"lambda_{idx:<3} | {actual:15.8f} | {'0.00000000':^16} | {'Converged':^12} | {err_str:^15}")
print("-" * 85)
print("\nConclusion: The number of iterations required depends on the 'spectral gap'. ")
print("Larger distances between subsequent eigenvalues lead to faster geometric convergence.")
print("=" * 85)

```

```

=====
QUESTION 2 (TASK 3e): ITERATIONS REQUIRED FOR 10^-7 ACCURACY
=====

```

Eigenvalue	Actual Value	Achieved Value	Iterations	Final Error
lambda_1	12.47020120	12.47020116	12	4.3095e-08
lambda_2	5.98130177	5.98130181	6	3.6238e-08
lambda_3	1.05483099	1.05483091	80	8.4280e-08
lambda_4	0.95881690	0.95881699	1	9.1137e-08
lambda_5	0.00000000	0.00000000	Converged	N/A (Zero Subspace)
lambda_6	-0.00000000	0.00000000	Converged	N/A (Zero Subspace)

```

=====

```

Conclusion: The number of iterations required depends on the 'spectral gap'.
Larger distances between subsequent eigenvalues lead to faster geometric convergence.

